

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME: Cloud Computing and
Big Data Analytics

COURSE CODE: CSA1522

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.*
(BL4)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

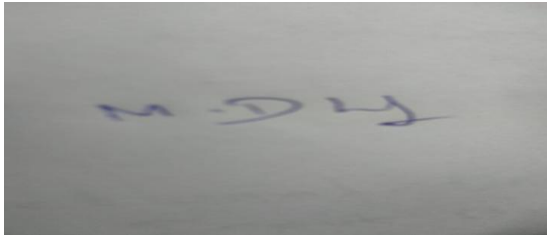
Total Marks: 30

Code of Conduct

I DHILIBAN M/192421166 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Assessment Tasks

Reverse Engineering Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.
2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.
3. A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing

structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.

4. A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.
5. A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

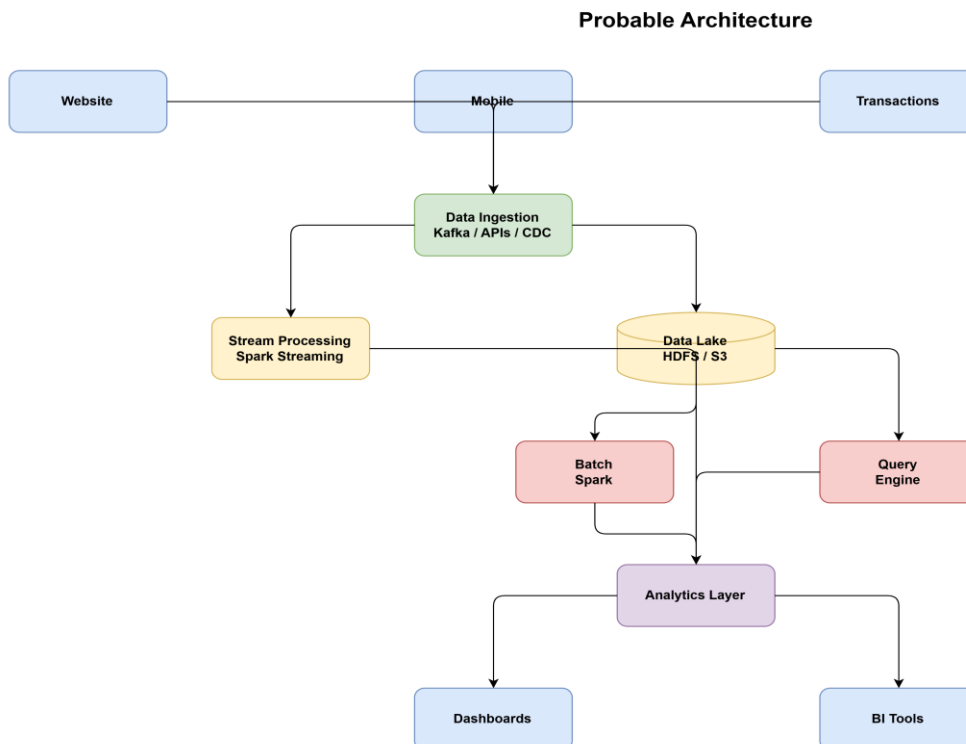
Reverse Engineering Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Inference Accuracy	25%	Infers system components and flow with high accuracy	Mostly accurate inference	Basic inference	Weak inference
Understanding of Architecture	25%	Shows clear understanding of underlying big data architecture	Good understanding	Partial understanding	Poor understanding
Use of Technical Evidence	20%	Uses strong reasoning and technical evidence	Reasonable evidence	Limited evidence	No meaningful evidence
Depth of Analysis	15%	Analysis is deep and insightful	Reasonably deep	Basic depth	Superficial analysis
Clarity	15%	Clear and organized explanation	Generally clear	Somewhat unclear	Poorly presented

1. Website Clicks, Mobile Interactions and Customer Transactions

1. Problem Analysis

The platform handles billions of events every day, including website clicks, mobile events, and customer transactions. This indicates a **distributed, highly scalable architecture** with both batch and real-time processing.



Distributed Storage

A distributed file system such as **HDFS** or cloud object storage such as Amazon S3 is likely used.

- Raw clickstream data can be stored in a data lake.
- Structured transaction data can be stored in a warehouse or distributed SQL database.
- Historical data is partitioned by date, region, customer, or event type.

Data Separation

Structured data:

- Customer ID
- Product ID

- Transaction amount
- Date
- Payment information

This can be stored in relational databases or data warehouses.

Semi-structured data:

- JSON mobile events
- Web logs
- Clickstream events

These are better suited to a data lake and schema-on-read processing.

Ingestion and Processing

Apache Kafka is a likely ingestion framework because it can handle very high volumes of continuous events.

Apache Spark can perform large-scale batch processing.

Spark Structured Streaming can process real-time events.

Scalability and Partitioning

Kafka topics can be divided into multiple partitions so several consumers process data simultaneously.

HDFS/S3 data can be partitioned by:

Year → Month → Day → Region → Event Type

This reduces the amount of data scanned during queries.

Replication provides fault tolerance if a storage node fails.

Analytics and Dashboards

Processed data can be exposed through SQL engines such as **Trino/Presto** or a cloud data warehouse.

BI tools such as Power BI or Tableau can connect to the analytics layer to display:

- Sales
- Customer activity
- Conversion rates
- Revenue
- Traffic
- Real-time KPIs

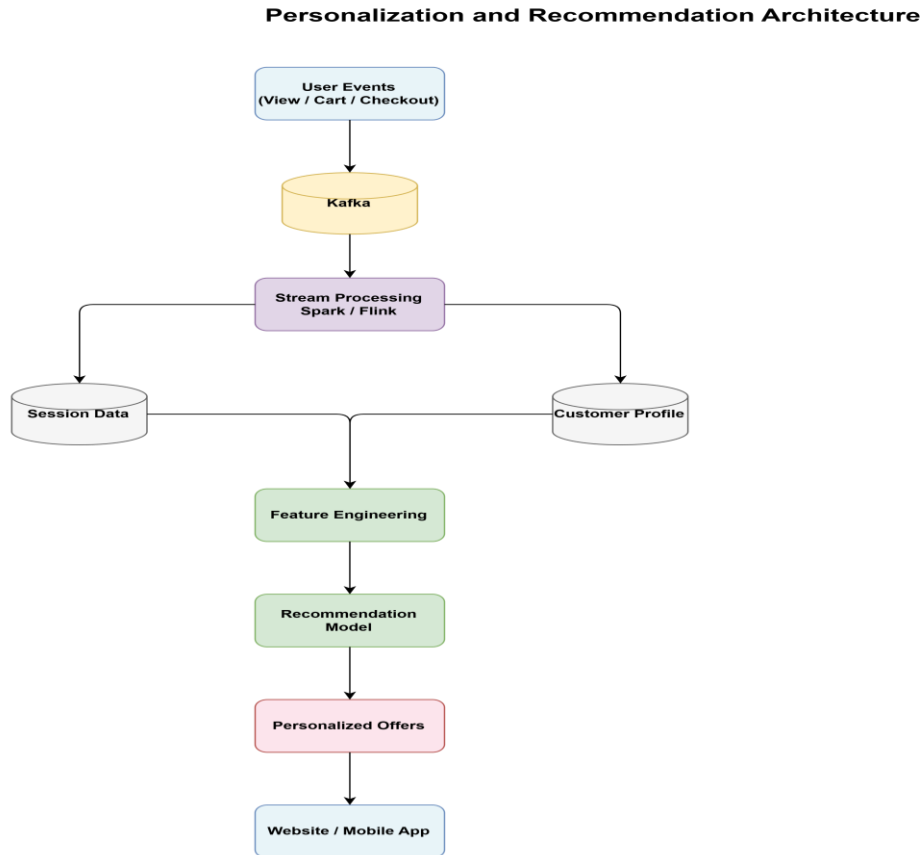
Reverse Engineering Conclusion

The evidence strongly suggests a **lambda/kappa-style architecture** combining distributed storage, Kafka-based ingestion, Spark processing, partitioning, replication, and BI tools.

2. E-Commerce Customer Behavior and Recommendations

Problem Analysis

The platform tracks product views, cart additions, abandoned carts, checkout behavior, and payments. These events need to be processed quickly to personalize recommendations.



Event Streaming

Apache Kafka is likely used because cart, view, and payment events arrive continuously.

A streaming framework such as **Apache Flink** or Spark Structured Streaming can calculate:

- Current session
- Recently viewed products
- Cart status
- Abandoned checkout
- Customer activity

Customer Profile Database

A NoSQL database such as **Cassandra, DynamoDB, or MongoDB** may store rapidly changing customer profiles.

A distributed cache such as **Redis** can store frequently accessed information.

For example:

Customer 101

└ Recently Viewed: Laptop, Mouse

└ Cart: Laptop

└ Previous Purchases: Keyboard

└ Recommended: Laptop Bag

Data Synchronization

Structured order data can come from transactional databases using **CDC (Change Data Capture)**.

Browsing events arrive through Kafka.

A stream-processing layer joins these sources to create a real-time customer profile.

Recommendation Workflow

1. Collect customer events.
2. Build customer behavior features.
3. Combine browsing and purchase history.
4. Train recommendation models.
5. Generate recommendations.
6. Cache recommendations.
7. Display them immediately to the customer.

Possible models include:

- Collaborative filtering
- Content-based recommendation
- Matrix factorization
- Gradient boosting
- Deep learning

Query and Caching

Trino/Presto can provide distributed analytical queries.

Redis can cache:

- Product information
- Customer profiles
- Popular products
- Recent recommendations

This reduces database latency.

Fault Tolerance and Scalability

Kafka replication protects event streams.

Distributed databases replicate customer profiles.

Multiple Spark/Flink workers allow horizontal scaling.

KPI Dashboard

The analytics layer can generate:

- Conversion rate
- Cart abandonment rate
- Average order value
- Revenue
- Product views
- Recommendation click-through rate

Conclusion

The likely design is a **real-time personalization pipeline** combining Kafka, stream processing, NoSQL, Redis, distributed queries, and machine learning.

3. Healthcare System – Patient Histories, Images, Sensors and Prescriptions

Problem Analysis

Healthcare data contains structured records, unstructured medical images, and continuously generated wearable data. Therefore, a **hybrid storage architecture** is likely required.



Hybrid Database Architecture

Structured data:

- Patient demographics
- Prescriptions
- Lab results
- Appointment records

Can be stored in relational databases such as PostgreSQL or healthcare data warehouses.

Unstructured data:

- X-rays
- CT scans
- MRI images
- Medical documents

Can be stored in secure object storage or specialized medical imaging systems.

Data Integration

Healthcare organizations commonly require interoperability standards such as:

- **HL7**
- **FHIR**
- **DICOM** for medical images

APIs can connect hospitals, laboratories, insurance systems, and clinical applications.

Streaming and Machine Learning

Wearable sensors continuously produce:

- Heart rate
- Blood oxygen
- Temperature
- Activity
- Sleep information

Kafka can ingest these streams, while Spark Streaming or Flink can process them.

Machine learning can support:

- Disease-risk prediction
- Patient deterioration detection
- Readmission prediction
- Abnormal vital-sign detection

Privacy and Security

Healthcare information is highly sensitive. The system would likely implement:

- Encryption at rest
- Encryption in transit
- Role-Based Access Control
- Multi-factor authentication
- Data masking
- Audit logging
- Data anonymization
- Secure APIs

Clinical Analytics

Patient Data

|



Data Integration

|



Data Cleaning

|



Feature Engineering

|



ML Models

|



Risk Score / Clinical Insight

|



Doctor / Healthcare Dashboard

Example

A patient's wearable device continuously reports abnormal heart-rate patterns. A streaming model can identify the abnormality and generate an alert for clinical review.

Conclusion

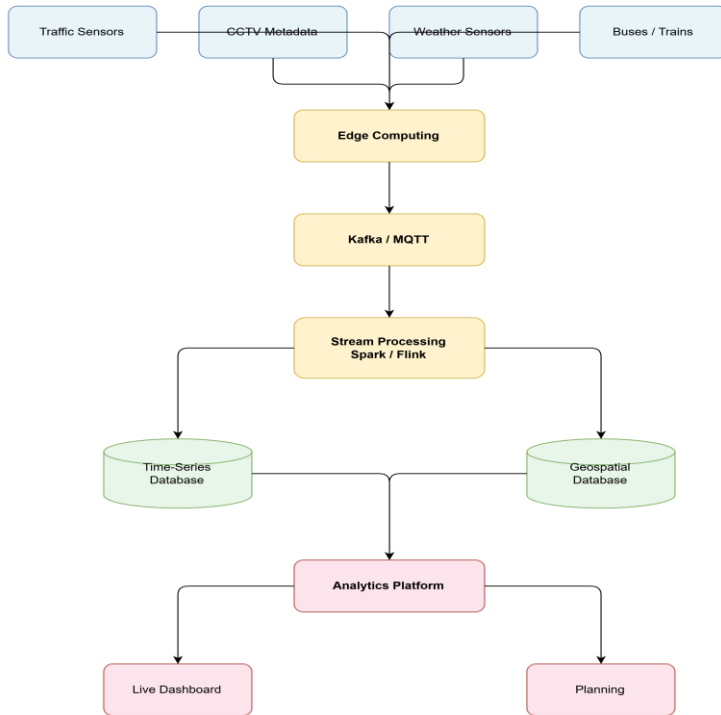
The probable solution is a **secure hybrid healthcare data platform** combining relational databases, object storage, Kafka, Spark, FHIR/HL7, DICOM, and machine learning.

4. Smart City – Traffic, CCTV, Weather and Public Transport

Problem Analysis

A smart city continuously generates sensor, transportation, weather, CCTV metadata, and geospatial data. Because many decisions are time-sensitive, edge and stream processing are important.

Probable Architecture



Edge Computing

Edge devices can process data close to where it is generated.

For example, cameras can perform initial object detection at the edge instead of sending every video frame to a central server.

This reduces:

- Network bandwidth
- Latency
- Central processing requirements

Messaging

MQTT is suitable for lightweight IoT sensor communication.

Kafka can handle high-volume event streaming between city systems and analytics platforms.

Time-Series and Geospatial Databases

Time-series databases such as InfluxDB or TimescaleDB can store:

- Vehicle speed
- Traffic volume
- Temperature
- Sensor readings

PostGIS can support location-based queries.

For example:

Find all traffic sensors

within 2 km of a congested road.

Streaming Analytics

Spark Streaming or Flink can calculate:

- Current traffic density
- Average vehicle speed
- Congestion levels
- Estimated travel time

Machine learning models can predict congestion based on:

- Historical traffic
- Weather
- Time of day
- Events
- Road conditions

Batch + Real-Time Analytics

Real-time: Detect current congestion and control traffic operations.

Batch: Analyze months or years of traffic information for road planning and infrastructure development.

Security

The platform would likely use:

- Encryption
- Authentication
- RBAC
- Network segmentation
- Secure APIs
- Audit logging
- Data anonymization
- CCTV access restrictions

Visualization

Tools such as Power BI, Tableau, Grafana, or custom dashboards can display:

- Live traffic maps
- Bus locations
- Congestion alerts
- Weather conditions
- Infrastructure status

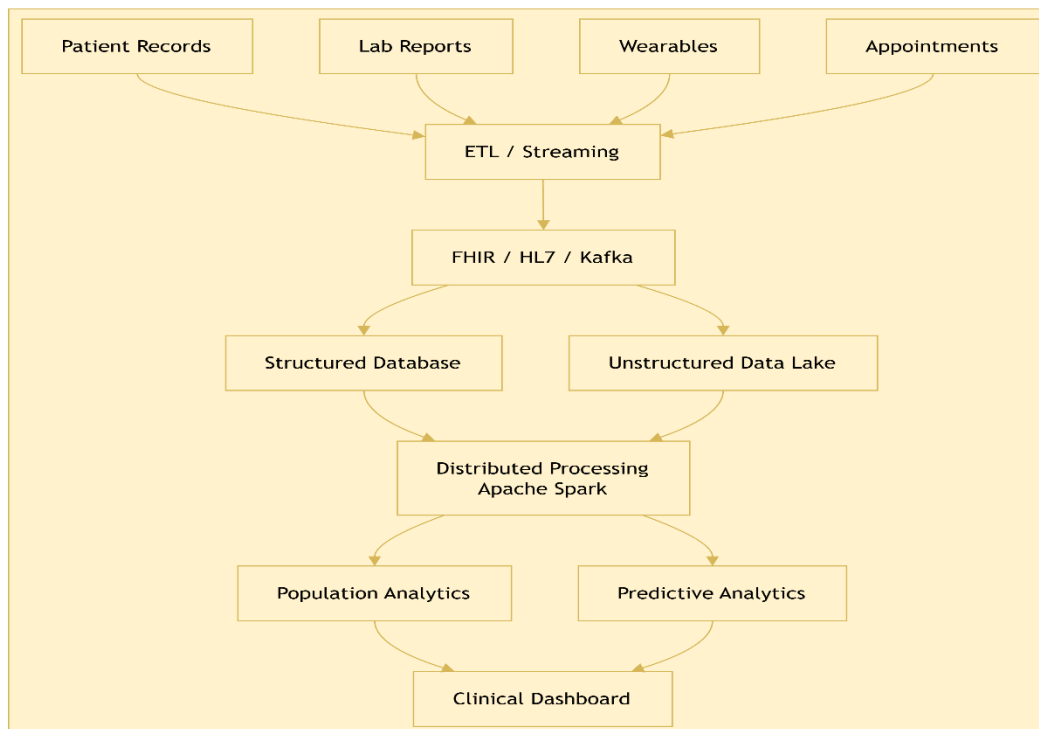
Conclusion

The architecture likely combines **edge computing + Kafka/MQTT + stream processing + time-series/geospatial databases + batch analytics** to provide real-time urban intelligence.

5. Healthcare Analytics – Patient Records, Wearables and Lab Reports

Problem Analysis

This system combines highly sensitive structured medical records with wearable sensor streams, laboratory information, and appointment histories. Therefore, it requires both **hybrid storage and secure distributed analytics**.



Hybrid Storage

Structured information such as:

- Patient ID
- Appointments
- Lab values
- Prescriptions

can be stored in relational databases.

Wearable data and medical documents can be stored in distributed storage or a cloud data lake.

ETL and Interoperability

ETL pipelines perform:

Extract → Transform → Validate → Load

Healthcare interoperability can use **HL7/FHIR** for clinical data exchange.

Wearable data can enter the system through APIs or streaming platforms.

Real-Time Monitoring

Kafka can ingest wearable readings.

Spark Structured Streaming or Flink can analyze the data.

For example:

Wearable

↓

Heart Rate

↓

Kafka

↓

Stream Processing

↓

Abnormality Detection

↓

Alert

Distributed Population Analytics

Apache Spark can analyze millions of patient records in parallel.

This can identify:

- Disease trends
- Risk factors
- Treatment outcomes
- Hospital readmission patterns
- Population health trends

Machine Learning

Possible models include:

Risk Classification: Predict whether a patient has a high disease risk.

Regression: Predict future health measurements.

Anomaly Detection: Identify abnormal wearable readings.

Recommendation Models: Assist clinicians by identifying potentially suitable treatment options based on approved clinical data and decision-support rules.

Security and Compliance

The platform should implement:

- Encryption at rest and in transit
- Role-based access
- Strong authentication
- Audit trails
- Data masking
- Anonymization for research
- Data retention policies
- Continuous security monitoring

Every access to sensitive patient information should be logged.

Example

A system could combine a patient's previous medical history, laboratory values, appointment records, and wearable readings to calculate a risk score. The score can be presented to a clinician as **decision support**, rather than replacing professional medical judgment.

Conclusion

The likely architecture is a **secure hybrid healthcare analytics platform** using relational storage, data lakes, Kafka, Spark, FHIR/HL7, encryption, audit mechanisms, and machine-learning models.